

Last updated: January 12, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 1

Tasks/Questions

- When you have first logged in to a shell, how (i.e., using what commands?) can you find out information about the environment? Consider the following as examples. Try to find out multiple ways for each (if you can).
 - The version of your Linux distribution and the version of your Linux kernel.
Among other ways: `lsb_release -a`, `cat /proc/version`, and `uname -a`
There are also files containing such information: e.g., `/etc/os-release`, and `/usr/lib/os-release`
 - RAM, remaining/total disk space, and CPU.
Among other ways: `free`, `df`, `cat /proc/cpuinfo`
 - The name (binary path) of the current shell.
`echo $SHELL`
- Using the `man` command, find out what the following commands do: `which`, `pwd`, `who`, `whoami`, `whereis`, and `whatis`.
- Linux commands can be classified as internal (built into the shell) and external (separate program binaries). How can you tell if a specific command (e.g., `cd`) is internal or external?
 - For the program you just compiled above, what if you used `hello` to run it instead of `./hello` (and why)?
Without a pathname `hello` will be interpreted as a command (either internal or external) so the system will say "not found". The dot (`.`) means the current directory so if that `hello` file is in your home directory "`./hello`" will be equivalent to "`/home/student/hello`".
 - Figure out where at least three external commands reside on the system.
Check if the output of this is empty (then it's internal): `which <command>`
As long as it is not a program binary file, it is part of the shell functionality (the shell does what you expect from the command).
- Making your own commands: the `PATH` environment variable lists the directories the shell uses to search for external commands. Where can you find documentation on it? How can you add the current directory (whichever directory you are currently in) to `PATH`? Then, how to make that change permanent? Try to identify multiple ways. You can now revisit the question about "hello" above.
You are making your own commands when you add the path of your command binary to `$PATH`. Specifically, you can append something to the existing `PATH` variable (e.g., `/home/student`):
`export PATH=$PATH:/home/student`
To make it permanent (so that you don't need to type the command above every single time before you use it), you can get it executed automatically by either the OS or the shell. In the case of `bash`, there are files like (to name a few) `~/.bash_profile`, `~/.bash_login` and `~/.bashrc`, you can put this line in. Note: you do not need to do this to make it permanent unless you

really want to add a command.

5. Look at the permissions of the program binaries of the external commands you have just found above. Who owns them? What group are they in?
6. For those same program binaries, figure out what the permission bits mean by reading the man page of `chmod` (this is the command you could use to change those permission bits).
Refer to the explanation of permissions in the instructions. We will also discuss this in class.
7. What are the owner, group, and permissions of `/etc/passwd` and `/etc/shadow`? What are these files used for (and how did you find this out)?
These are very important files, one storing user account info, one storing the hashed passwords, which will be discussed in more detail later.
8. What does it mean to have execute permission on a directory?
Whether you can enter (`cd` into) that directory.
9. The `ls` command can be used to get a listing of the files in a directory. What options are passed to `ls` to see: the permission bits above; all the files within a directory (including hidden files)? How to make a file hidden?
You can use the options “`-l`” and “`-a`”. Any file/directory with a name starting with a dot (`.`) character is hidden.
10. Compile and run `csimpleshell.c`. How does its functionality compare to that of `bash`? List at least 3 differences.
There should be many differences as it merely does very little. Examples: 1) if you want to hit Tab to autocomplete typing a command, it will not work here. 2) It does not support operators like `&` (to run in background) and `>` (redirection). 3) It does not have syntax highlighting, i.e., all text in the same color. 4) You cannot use the up and down arrows to browse previously used commands.